



INVENTIX · TEAM 4

Backend build report

Everything that is not writing code: the commands run, the dependencies chosen, the decisions taken, the two migrations the backend needed, and what the verification proved.

Source docs/bereport.md

Companion 08-api-contract.md

Generated 3 September 2026

Contents

1. In one paragraph

2. What now exists

3. Dependencies

4. Decisions taken while building

The Supabase client, not a direct Postgres connection

Where the transactions went

The role is read from the database, not the token

ML files are interfaces that admit they cannot answer

5. Two migrations the backend needed

0017_create_order.sql — a function, not a table

0018_job_tracking_columns.sql — two columns on orders

6. What was run

The three problems found, and what they were

7. What the verification proved

It starts and serves

Every promised endpoint exists

The security layer actually works

114 tests, and what they are about

8. What was not done

9. How to run it

10. What has to happen before it does anything real

11. Documents updated

Everything that is not "I wrote a Python file": the commands run, the dependencies chosen, the decisions taken, the two database migrations the backend needed, and what the verification proved.

1. In one paragraph

Every stub in `backend/` is now real code: 77 files, 5,662 lines, implementing all 41 endpoints in `docs/08-api-contract.md`. The application starts, serves interactive documentation, and rejects forged and expired sign-ins correctly. 114 unit tests pass and the linter is clean. Two migrations were added because the backend needed database objects that did not exist. Nothing was integrated with the frontend, as asked -- not one file under `frontend/` was touched.

2. What now exists

Folder	Files	Lines	Contents
<code>app/core/</code>	5	322	Errors, token verification, Supabase clients, idempotency, logging
<code>app/domain/</code>	7	686	The business rules. No HTTP, no SQL
<code>app/feeds/</code>	39	2,841	Twelve feeds, three files each, plus the six-file upload pipeline
<code>app/integrations/</code>	4	249	WhatsApp, email, Firebase, retry queue
<code>app/jobs/</code>	7	434	The six scheduled jobs from spec 14, and the scheduler
<code>app/ml/</code>	5	189	Interfaces only. Spec 7.2 builds these last
top level	4	300	<code>main.py</code> , <code>api.py</code> , <code>config.py</code> , <code>dependencies.py</code>
<code>tests/</code>	7	642	114 tests over <code>domain/</code> and the upload parser

Measured from the running application: **41 endpoints across 36 paths**, plus `/health`.

3. Dependencies

Pinned exactly, so every team member and the marker build the same thing. Python 3.14.6.

Package	Version	Why
<code>fastapi</code>	0.141.1	The framework
<code>uvicorn[standard]</code>	0.52.4	The server
<code>pydantic</code>	2.13.5	Request and response validation from type hints
<code>pydantic-settings</code>	2.15.0	Typed environment variables in one place
<code>supabase</code>	2.31.0	The only way this backend reaches the database
<code>PyJWT[crypto]</code>	2.13.0	Verifying the Supabase access token
<code>httpx</code>	0.28.1	Outbound calls to WhatsApp and email
<code>apscheduler</code>	3.11.3	The scheduled jobs in spec 14
<code>pandas</code> + <code>openpyxl</code>	3.0.5 / 3.1.5	Parsing CSV and Excel sales reports
<code>python-multipart</code>	0.0.32	Receiving the uploaded file
<code>firebase-admin</code>	7.5.0	Push notifications
<code>pytest</code> , <code>pytest-asyncio</code> , <code>ruff</code>	9.1.1 / 1.4.0 / 0.16.5	Tests and linting

Two are deliberately not installed. `scikit-learn` and `numpy` are commented out in `requirements.txt`. Spec 7.2 builds the machine learning last because it needs sales history that does not exist yet, so installing 200 MB of libraries that nothing imports would only slow every build. The comment says when to uncomment them.

The install pulled 78 packages in total once transitive dependencies were resolved.

4. Decisions taken while building

The Supabase client, not a direct Postgres connection

`requirements.txt` already named `supabase`, and `core/supabase.py` 's header already described "two clients: anon for user-scoped reads, service-role for backend writes". That was followed rather than substituted, and it turned out to be the better design for one reason worth stating:

The backend acts as the signed-in user by default. `CurrentUser.db` builds a client carrying the caller's own token, so row level security applies to every query the backend makes on their behalf. A bug in a service filter cannot leak another shop's rows, because the database refuses independently. The service-role client -- which bypasses RLS entirely -- is used in only three places: writing the `profiles` row at registration, the ranking job, and sending notifications.

Where the transactions went

PostgREST cannot span several writes in one transaction. Three operations in the specification genuinely need one:

Operation	Why it must be atomic
Creating an order	Spec 6.5 writes the order, its lines and the restock flags. A crash between them leaves an order with no lines, or products that can be ordered twice
Changing a quantity	Spec 5.10 requires the audit row to be inseparable from the change
Confirming receipt	Spec 11.4 tops up stock, writes audit rows and clears flags

So the transactions live in the database as SQL functions, the same reasoning that put `apply_stock_adjustment` there in the first place. `create_order` is new -- see section 5.

The role is read from the database, not the token

`dependencies.get_current_user` verifies the token, then reads `profiles.role` with one query. Supabase does not put an application role in the token by default, and spec 15.2 forbids trusting anything the app sends. That costs one query per request. A custom access token hook would remove it, and `core/security.py` says so in a comment -- it is the one file that would change.

ML files are interfaces that admit they cannot answer

`app/ml/` returns `None` and `is_available() -> False` rather than plausible-looking numbers. A fabricated forecast is worse than none, because a shop owner would order against it. Each file documents what lands there in phase 10 and what data it needs first.

5. Two migrations the backend needed

You asked to be told when tables change. Neither adds a table; both add to `orders`.

`0017_create_order.sql` — a function, not a table

Creates the order, its lines and the restock flags in one transaction. Section 4 explains why it cannot be three separate calls from Python.

`0018_job_tracking_columns.sql` — two columns on `orders`

Column	Set by	Without it
<code>unanswered_notified_at</code>	<code>jobs/unanswered_orders.py</code>	The hourly job re-notifies both parties every hour until someone answers
<code>auto_confirm_warned_at</code>	<code>jobs/auto_confirm.py</code>	The daily warning goes out every day rather than once

The second one matters more than it looks. A user who gets the same notification every hour stops reading notifications — and then misses the low-stock warnings too, which is the whole point of the application. Both columns are recorded on the order rather than inferred from the notifications table, so the job's query stays on one table with one index.

Both need running against Supabase alongside 0001 – 0016 .

6. What was run

Nothing touched a real Supabase project — there still isn't one.

#	Command	Purpose	Result
1	<code>pip index versions ...</code> × 15	Find current versions to pin	—
2	<code>python -m venv .venv</code>	Isolate from the system Python	Created
3	<code>pip install -r requirements.txt</code>	Install	78 packages, no conflicts on 3.14
4	<code>pytest</code> <code>tests/test_order_state_machine.py</code>	First check, before building on it	27 passed
5	<code>pytest tests/test_stock.py</code>		19 passed
6	<code>pytest tests/test_ranking.py</code>		18 passed
7	<code>pytest tests/</code>	Full suite	114 passed
8	<code>python -c "from app.main import app"</code>	Does it import at all	Failed — import depth
9	(fix)	Four files reaching <code>app.feeds.core</code>	Fixed
10	<code>app.openapi()</code>	Count the real endpoints	42 operations
11	compare against <code>docs/08-api-contract.md</code>	Does it match what was promised	Every documented endpoint built
12	<code>ruff check app tests</code>	Lint	11 findings
13	<code>ruff check --fix</code>		4 auto-fixed
14	(fix)	7 remaining, including a real bug	Fixed
15	<code>ruff check app tests</code>		All checks passed
16	<code>uvicorn app.main:app --port 8123</code>	Start it	Serving
17	<code>curl /health</code> , <code>/docs</code> , <code>/customer/stocks</code>	Does it answer	200 / 200 / 401
18	mint three JWTs, call with each	Does the security actually work	See section 7

The three problems found, and what they were

Import depth (step 8). Four files in `app/feeds/shared/*` used `from ...core` — three dots from `app.feeds.shared.notifications` lands on `app.feeds.core` , which does not exist. Needed four. The application would not start at all. Found in two seconds by trying to import it; would have been found in about the same time by running it, and never by reading it.

A real timezone bug (step 14). `ruff` flagged `date.today()` in the seasonal code. It is right, and it is not a style point: `date.today()` reads the **server's** date. On a UTC host, between midnight and 05:30 Colombo time it returns *yesterday* — so a festival warning fires a day early or late, silently. Fixed with `seasonal.today()`, which reads the date in `Asia/Colombo`. That is the kind of bug that is nearly impossible to notice by testing locally in Sri Lanka.

Three lint findings that were wrong and are now annotated with the reason: the blind `except` in `config_store` is deliberate (config must degrade, not take the Suppliers screen down), the naive `strptime` is correct (a sale date carries no timezone), and `File(...)` in a default is FastAPI's own documented idiom.

7. What the verification proved

It starts and serves

```
GET /health -> {"status":"ok","environment":"development","database_configured":false}
GET /docs -> 200
```

`database_configured: false` is correct and deliberate. The application starts without Supabase so the API can be reviewed and the routes checked before the database exists; endpoints that need data return a clear 503 rather than a stack trace.

Every promised endpoint exists

41 endpoints compared against `docs/08-api-contract.md` programmatically, not by eye. Every documented endpoint is built. One endpoint was built that the contract did not list — `GET /supplier/delivery`, the queue already grouped by stage — and the contract has been updated rather than left disagreeing with the code.

The security layer actually works

Three tokens were minted and sent at a protected endpoint:

Token	Result
No token at all	401 "You need to be signed in to do that."
Expired an hour ago	401 "Your session has expired. Please sign in again."
Valid shape, signed with a different secret	401 "Your sign-in could not be verified."
Valid and correctly signed	503 — passed authentication, reached the database layer

The third row is the one that matters. A forged token with a perfectly well-formed payload is rejected, because the signature is checked against the real secret. The fourth proves the chain runs end to end: verification passed, the role lookup was attempted, and it failed only where it should — at the database that does not exist yet.

114 tests, and what they are about

File	Tests	Covers
<code>test_order_state_machine.py</code>	27	Every legal transition in spec 11.2, and that a supplier cannot mark an order purchased
<code>test_stock.py</code>	19	The three-way classification, the at-or-below boundary, the negative guard
<code>test_ranking.py</code>	18	The new-supplier problem, rating confidence, quality outweighing price
<code>test_message_builder.py</code>	26	Every product present, totals, singular vs plural, edit detection
<code>test_uploads.py</code>	24	Date and number quirks, exact alias matching, the file hash

These need no database and no server, because `domain/` touches neither. That is the point of the folder existing, and it is why a second person could have written and tested it in parallel with the schema.

Three worth calling out:

- `test_stock.py` : **at the threshold is low, not in stock**. Spec 5.4 says "at or below". An off-by-one there means an item never becomes low and the entire application never triggers.
- `test_ranking.py` : **a new supplier beats a genuinely bad one**. An unrated supplier must outrank a one-star supplier with forty ratings, or the ranking rewards being bad for long enough to be measured.
- `test_uploads.py` : **03/09/2026 is 3 September**. A month-first reading does not fail — it silently files a month of sales under the wrong dates and then trains the forecast on them.

8. What was not done

The frontend was not touched. Not one file under `frontend/` was changed, as asked. The `useMock(feed)` switch discussed earlier is still not made; it is a five-minute change and it belongs to whoever starts the integration.

Nothing has run against a real database. Every service method is written against the Supabase client, and the query shapes come from `docs/09-database-build.md`, but none has executed against real rows. The 114 tests cover `domain/` and the parser — the parts with no database. **This is the largest untested surface in the project** and the first thing to exercise once Supabase exists.

Two `ponytail:` markers, both deliberate:

- `integrations/queue.py` — a failed WhatsApp or email send is reported and logged, not retried automatically. Spec 15.4 is satisfied in that the order still exists and the user is told the message is pending. The retry table's shape depends on what the channels actually return, which is not known until Business API access is approved (open question Q11).
- `feeds/customer/uploads/service.py` — parsed rows are held in memory between the three upload steps, so a server restart mid-upload loses them and the owner chooses the file again. Storing the file in Supabase Storage would survive that; not worth it until uploads are common.

Reports return sections only, as you asked — five section descriptors, each marked with whether that shop has the data for it yet, and no figures.

No rating submission endpoint. Still the gap flagged in the API contract, and still the one to fix early:

`RatingPrompt` collects a score and comment and discards them, and spec 12.1 makes ratings 40 percent of the ranking. Every month without it is a month of unrankable suppliers.

`stock_items.owner_id` has no role guard, while `supplier_listings.supplier_id` does. Noted in the database report and still true.

9. How to run it

```
cd F:\Projects\InventiX\Project\backend
.venv\Scripts\activate
uvicorn app.main:app --reload --host 0.0.0.0 --port 8000
```

Then open <http://localhost:8000/docs> — FastAPI generates interactive documentation from the code, and every endpoint can be called from that page. It is the fastest way to see what the backend does without the app being involved.

`--host 0.0.0.0` matters: the default binds to `127.0.0.1`, which only accepts connections from this PC. A phone could not reach it.

To run the tests:

```
.venv\Scripts\python.exe -m pytest tests\ -q
```

10. What has to happen before it does anything real

1. Create the Supabase project and apply `0001` – `0018` plus functions, policies and seeds.
2. Copy `backend/.env.example` to `backend/.env` and fill in `SUPABASE_URL`, `SUPABASE_ANON_KEY`, `SUPABASE_SERVICE_ROLE_KEY` and `SUPABASE_JWT_SECRET`. The JWT secret is under **Project Settings, API, JWT Settings** — without it every request is a 401.
3. Register one real user through the app, then insert their `profiles` row by hand, and call `GET /auth/profile` from `/docs` with their token. That single call exercises the whole chain: token verification, the role lookup, and a real database read.
4. Then work through the feeds from `/docs`, one at a time.

Only after that is the frontend integration worth starting — and at that point it is one environment variable plus the `useMock(feed)` change.

11. Documents updated

File	Change
<code>docs/08-api-contract.md</code>	Added <code>GET /supplier/delivery</code> , which was built but not documented
<code>backend/requirements.txt</code>	Pinned every version; commented out the ML libraries with the reason
<code>backend/.env.example</code>	Unchanged — it already listed every variable the code reads